

JSON, XML, AJAX и работа с API

Содержание урока

★ Форматы передачи данных

★ Формат JSON

★ Формат XML

★ Технология AJAX

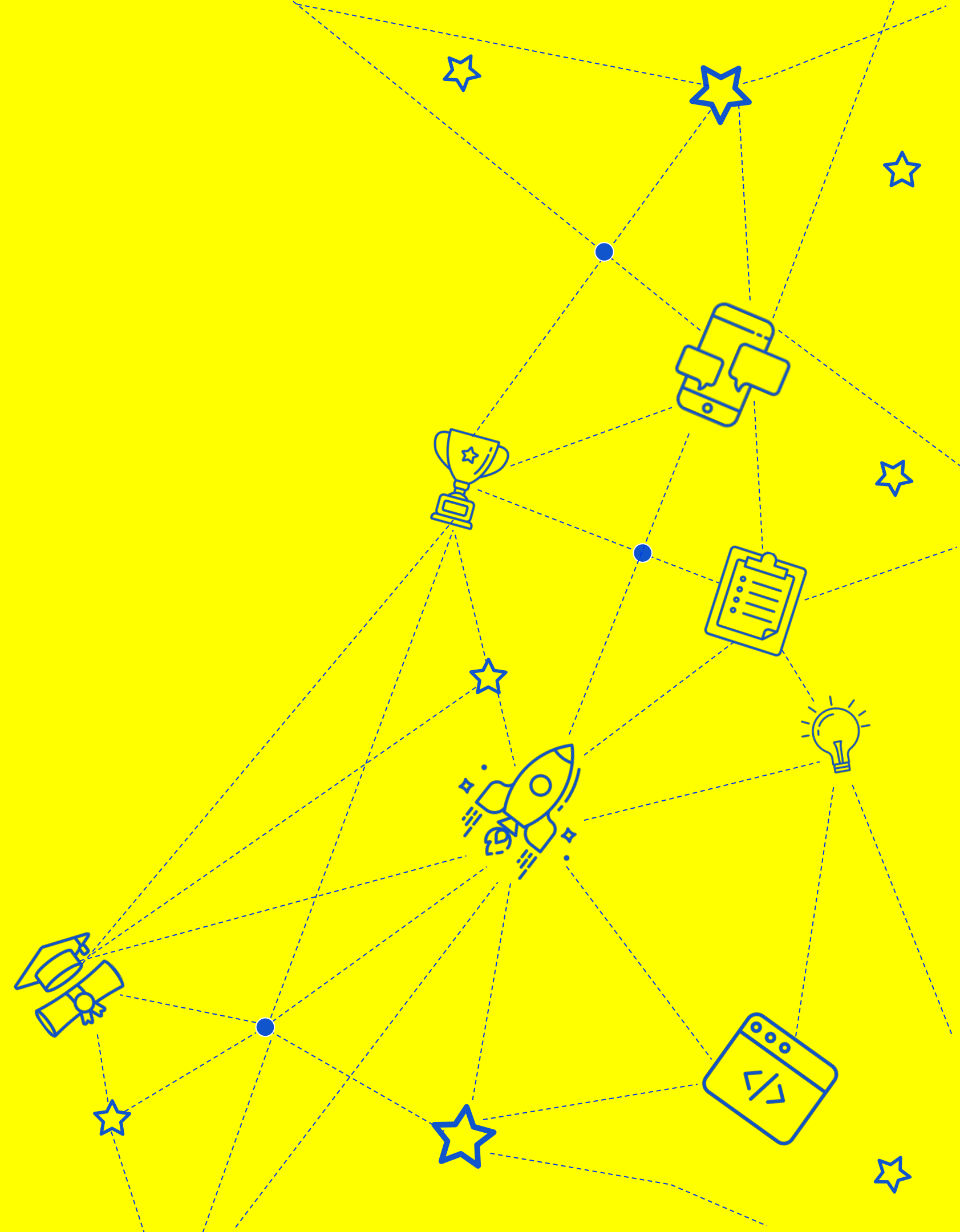


Senior Software Engineer

ИГОРЬ КРИВОНОС

- Проектирую и реализовываю web сервисы с 2014 года
- Консультирую стартапы по построению масштабируемой архитектуры приложения
- 7 лет активно занимаюсь менторингом

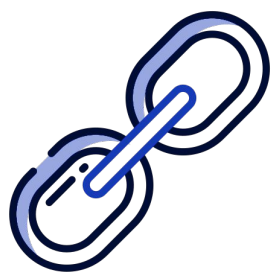
Форматы передачи данных



Что такое информация



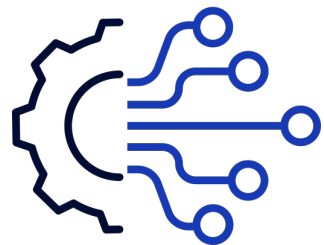
Как общаются приложения



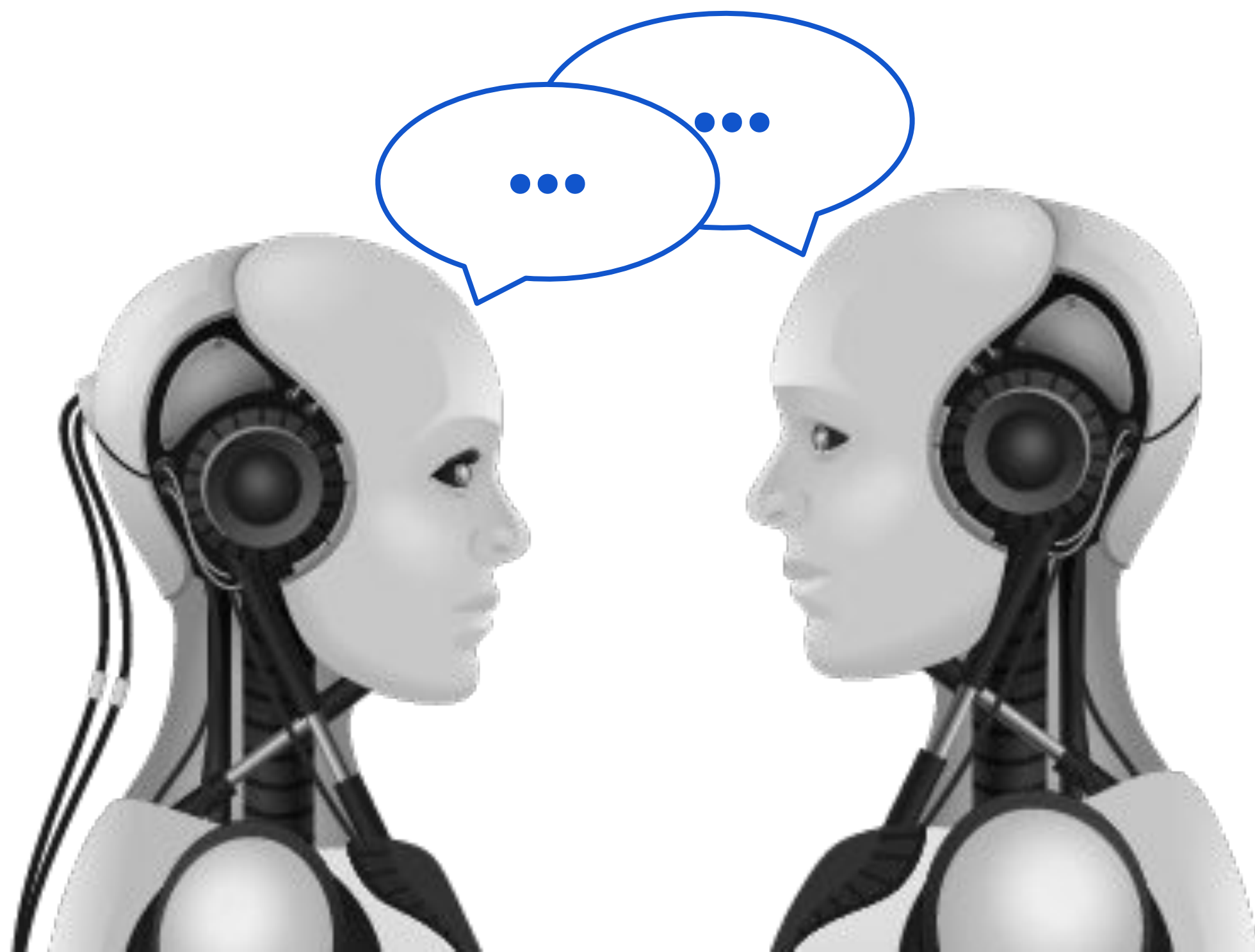
через http запросы



через socket



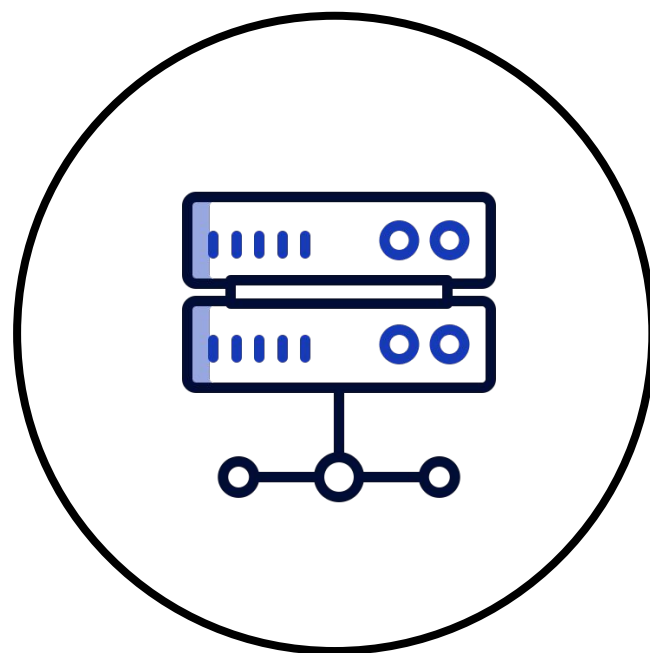
через web-hook



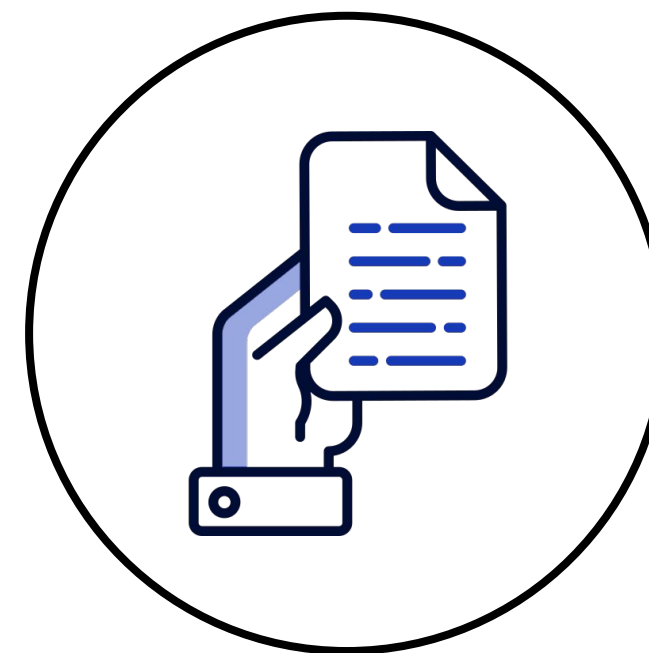
Как передать сообщение по сети?



Упаковать данные
в сообщение
для отправки

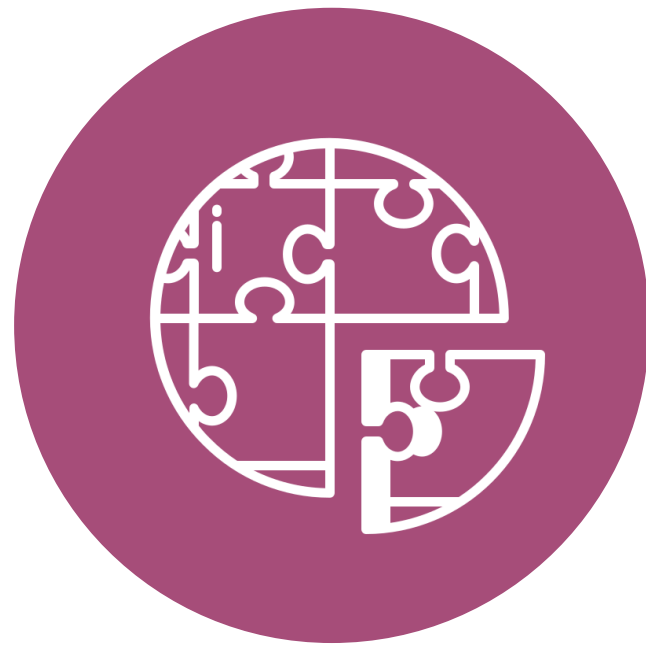


Переслать их
по сети

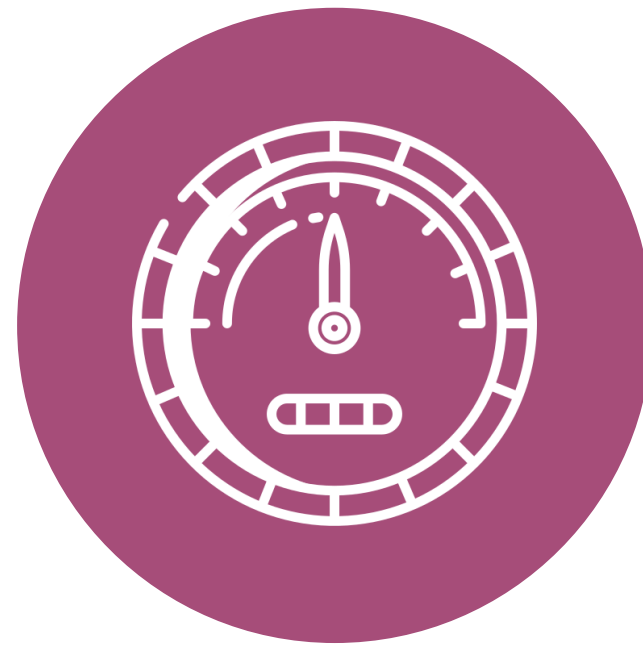


Распаковать
данные после
получения

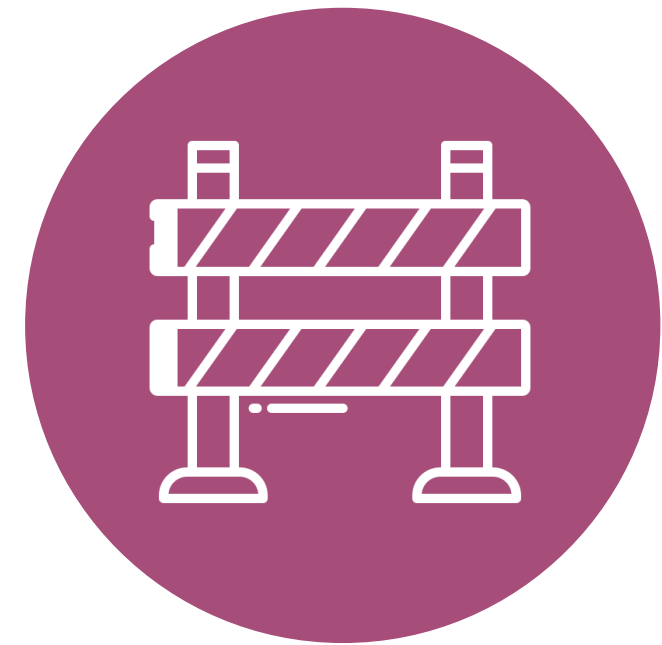
Почему нельзя передать данные напрямую?



Изолированность
отдельных
приложений

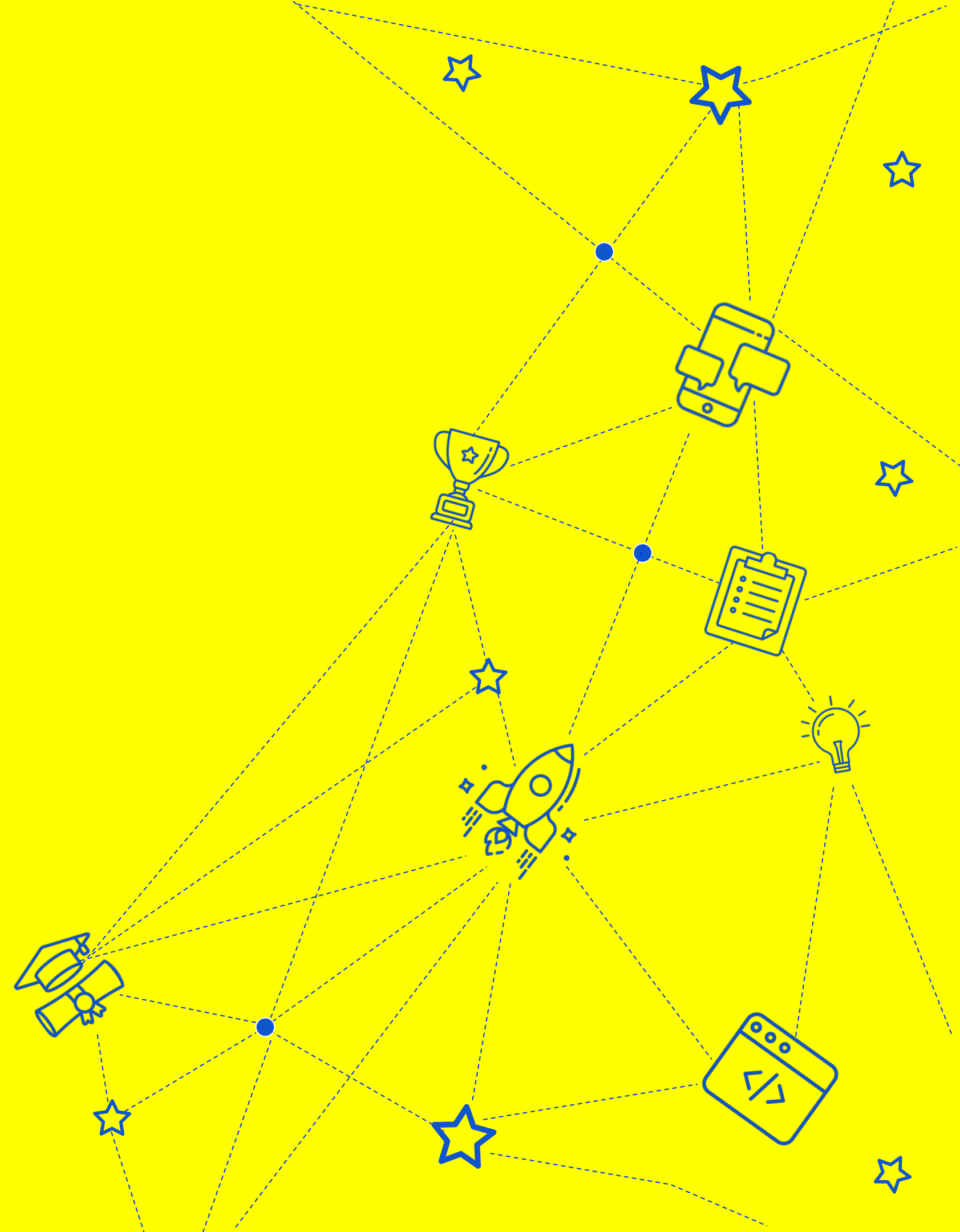


Скорость
передачи данных



Ограничения
протокола
передачи

JSON

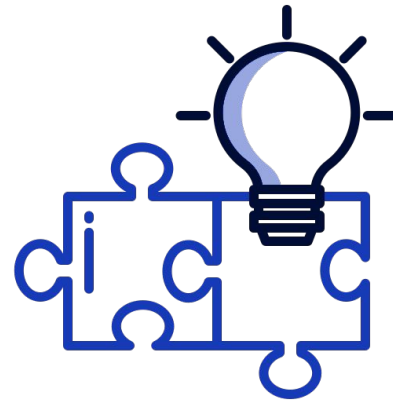


Типы данных



Простые:

- ★ цифры
- ★ текст
- ★ логические значения
- ★ null

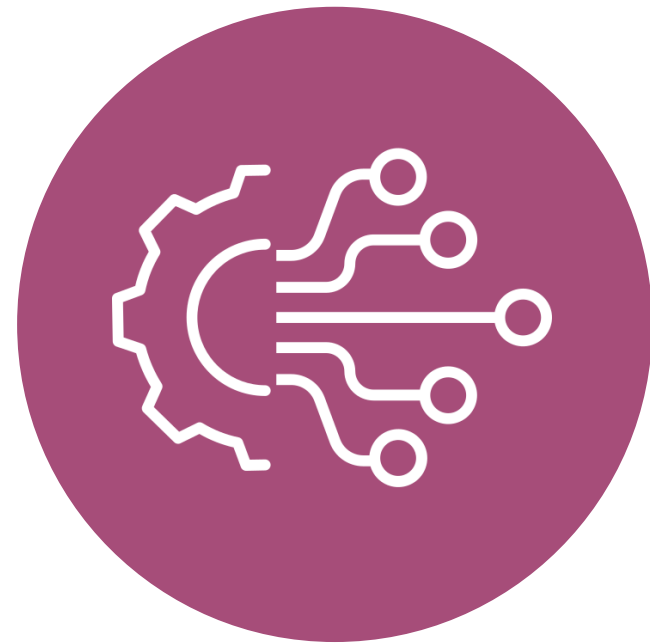


Сложные:

- ★ массив
- ★ объект



Что нельзя передать в JSON



Функции



Даты

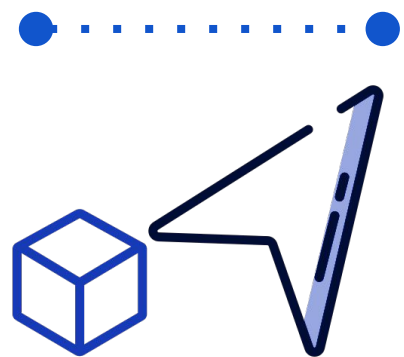


Классы



Картинку

А если очень хочется?



Сериализация



Пример

```
datetime(2000, 10, 18, 2, 5, 10)
```



```
"2000-10-18T02:05:10.000Z"
```

```
def multiply(a):  
    return a * 2
```



```
"def multiply(a):\n    return a * 2"
```



"/9j/4VpQRXhpZgAASUkqAAgAAAAMAA8BAGAGAAAANGAAABABAg
ANAAAApAAAABIBAwABAAAAAQAAABoBBQABAAAAxAAAABsBBQ
ABAAAAzAAAACgBAwABAAAAAgAAADIBAgAUAAAA1AAAADsBAGA
MAAAA6AAAABMCAwABAAAAAgAAAJiCAGABAAAAAAGmHBAA
BAAAAaAEAACWIBAABAAAA4iMAAOQqAABDYW5vbgBDYW5vbiBF
TlMgNkQAAAAAAAAAAAAAAAAAAAAAAAAAAAEgAAAABAAAASAAA
AAEAAAAYMDIzOjAxOjIIDE1OjMzOjEyAETHdGUgU2h5YnVuAAAAAA
AA
AA
AAAAAJgCaggUAAQAAADYDAACdggUAAQAAAD4DAAaiiAMAAQAA
AAEAAAAniAMAAQAAAIACAAAwIAMAAQAAAAIAAAAYIAQAAQAAAI
ACAAAKAcABAAAADAYMzADkAIAFAAAAEYDAAAEkAIAFAAAAFoD
AAABkQcABAAAAAECAwABkgoAAQAAAG4DAAACkgUAAQAAAHYD
AAAEkgoAAQAAAH4DAAAHkgMAAQAAAAIAAAAJkgMAAQAAABAA
AAAKkgUAAQAAAIYDAAB8kgcAaB4AAI4DAACGkgcACAEAAPYhAAC
QkgIAAwAAADc5AACRkgIAAwAAA...."

Простые типы



Числа

1
2
3

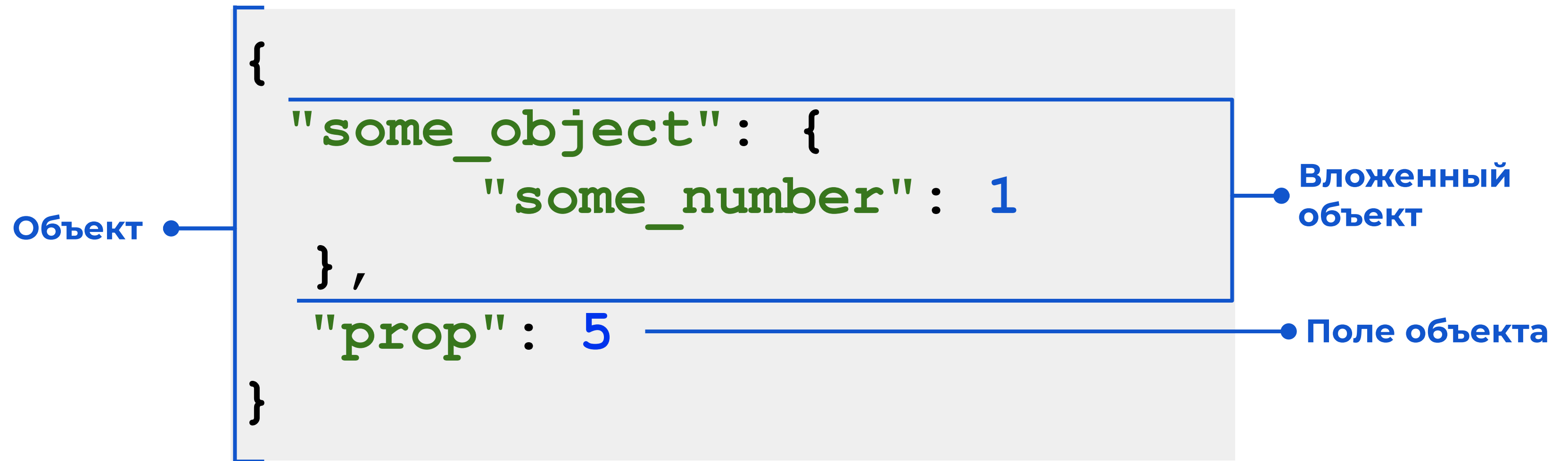
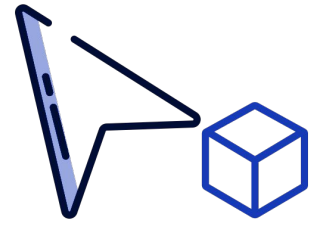
Строки

"some-string"
"some-another"

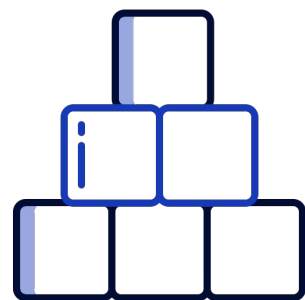
Логические значения

true
false

Объекты



Массивы



Массив

[

{

"some_number": 1

},

1,

"some string"

]

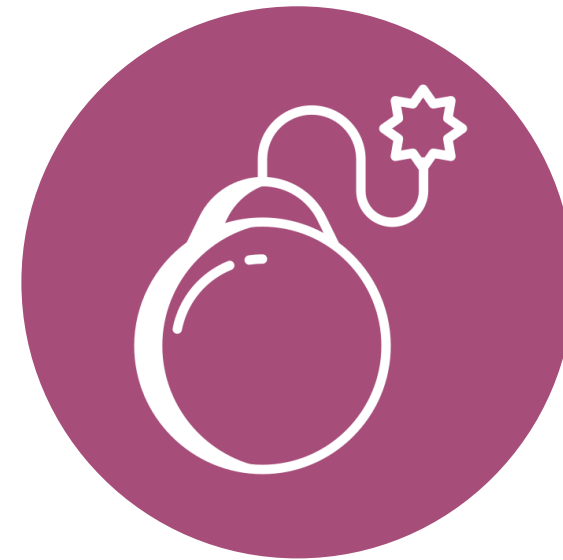
Элементы
массива

Плюсы и минусы



Плюсы:

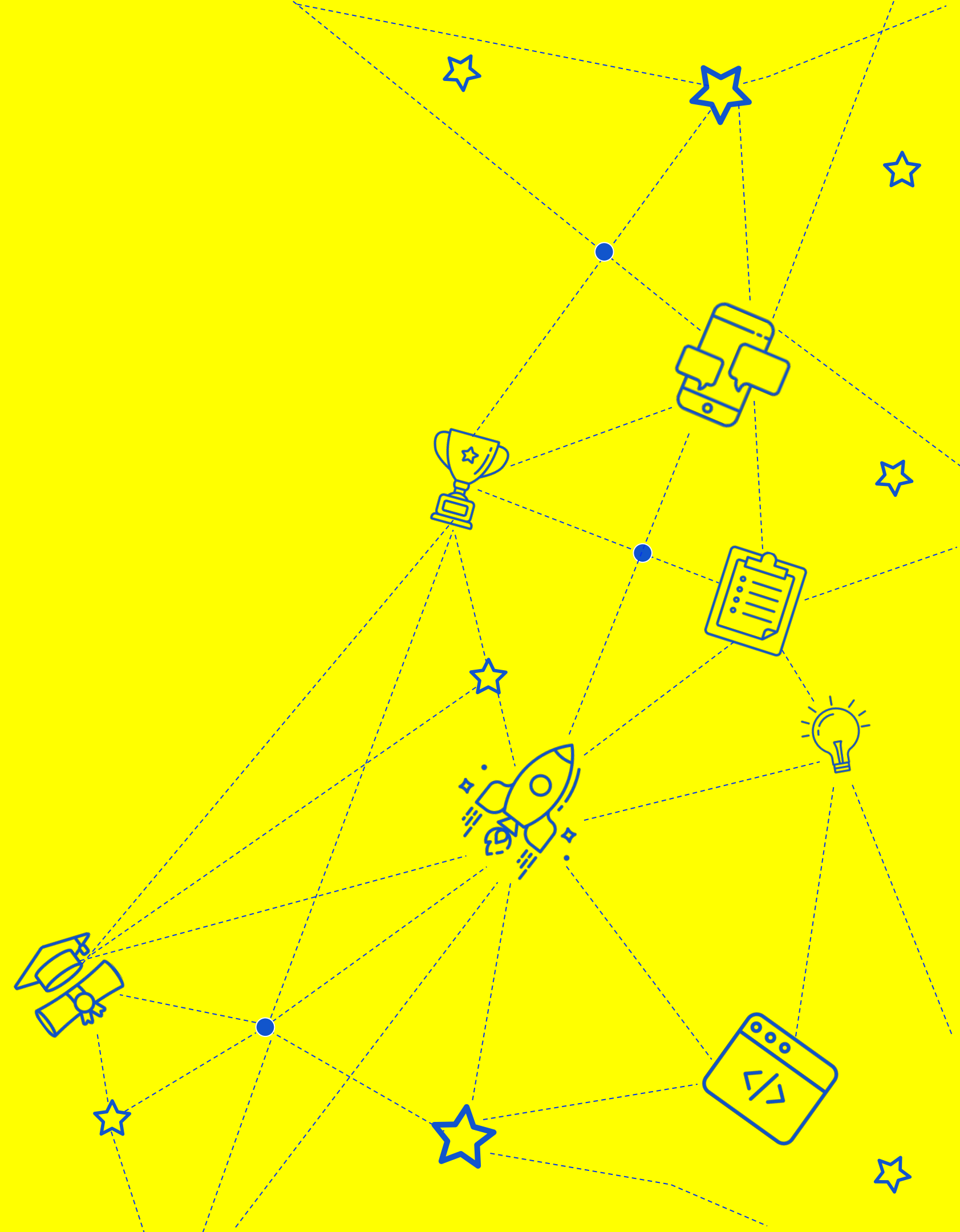
- ★ Строгий формат
- ★ Компактность
- ★ Распространенность



Минусы:

- ★ Мало типов данных

XML



Типы данных



Строка



Логические значения



Десятичные цифры



Цифры с плавающей запятой



16-ричные цифры



URI



Дата



Дата время



Время



Временной промежуток

Из чего состоит XML документ

Пролог

```
<?xml version="1.0"?>  
<!--File Name: Inventory.xml -->
```

Объявление XML

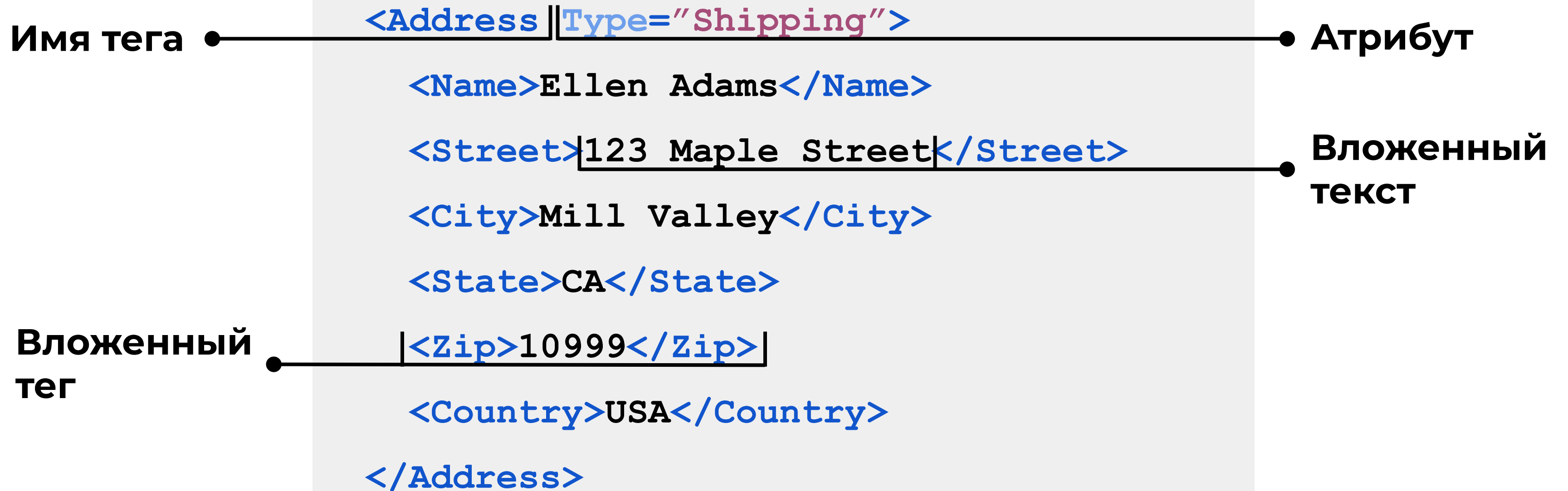
Комментарий

Элемент
Документ
(Корневой
элемент)

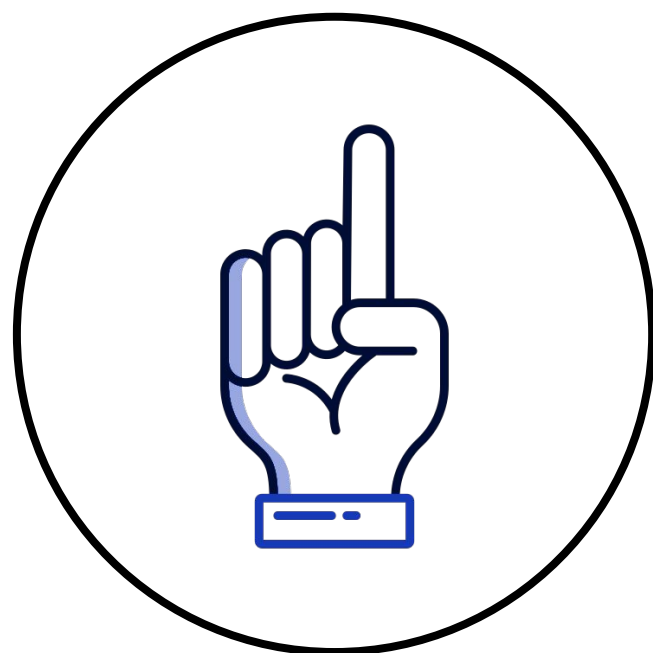
```
<INVENTORY>  
  <BOOK>  
    <TITLE>The Adventures of Huckleberry Finn</TITLE>  
    <AUTHOR>Mark Twain</AUTHOR>  
    <BINDING>mass market paperback</BINDING>  
    <PAGES>298</PAGES>  
    <PRICE>$5.49</PRICE>  
  <BOOK>  
    .  
    .  
    .  
  <BOOK>  
    <TITLE>The Turn of the Screw</TITLE>  
    <AUTHOR>Henry James</AUTHOR>  
    <BINDING>trade paperback</BINDING>  
    <PAGES>384</PAGES>  
    <PRICE>$3.35</PRICE>  
  <BOOK>  
</INVENTORY>
```

Элементы,
вложенные
в элемент
Документ

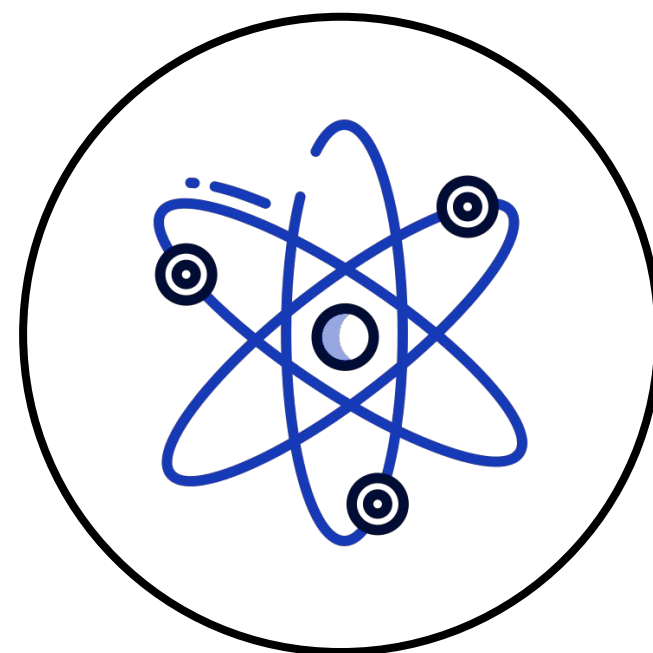
Из чего состоит тег



Чем XML отличается от HTML



XML имеет более строгие
правила синтаксиса



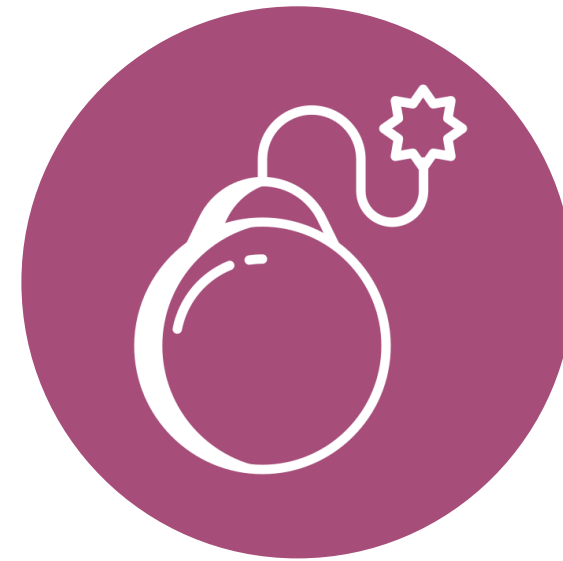
XML не имеет жестких
требований к именам тегов
и их атрибутам

Плюсы и минусы



Плюсы:

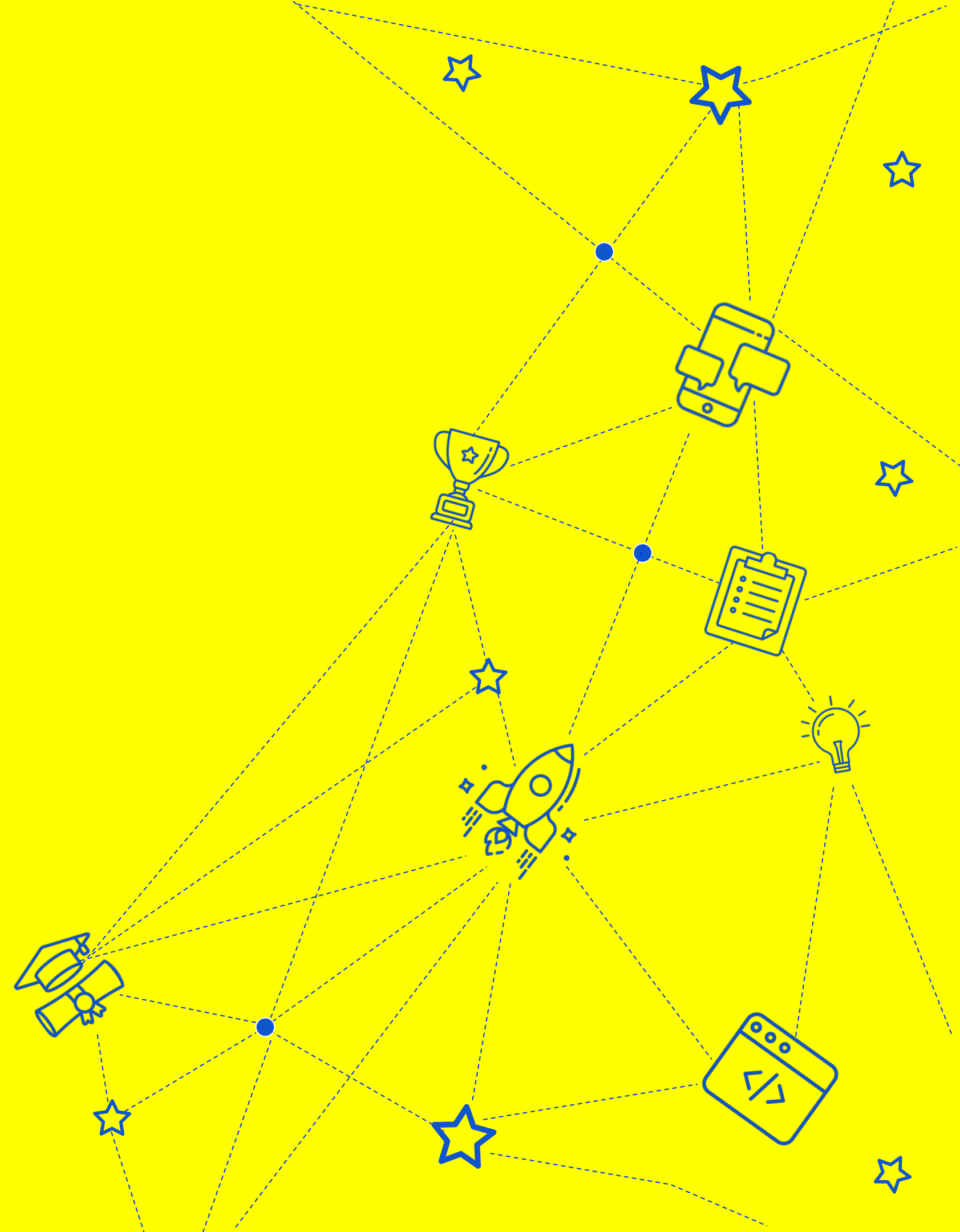
- ★ Строгий формат
- ★ Множество типов данных
- ★ Легко ориентироваться в большом документе



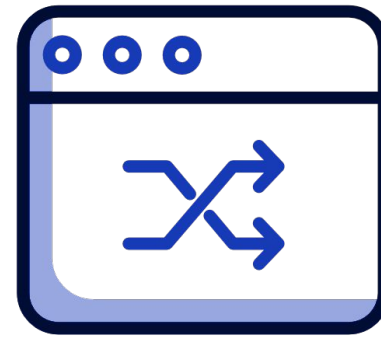
Минусы:

- ★ Большой размер

AJAX



Что это такое

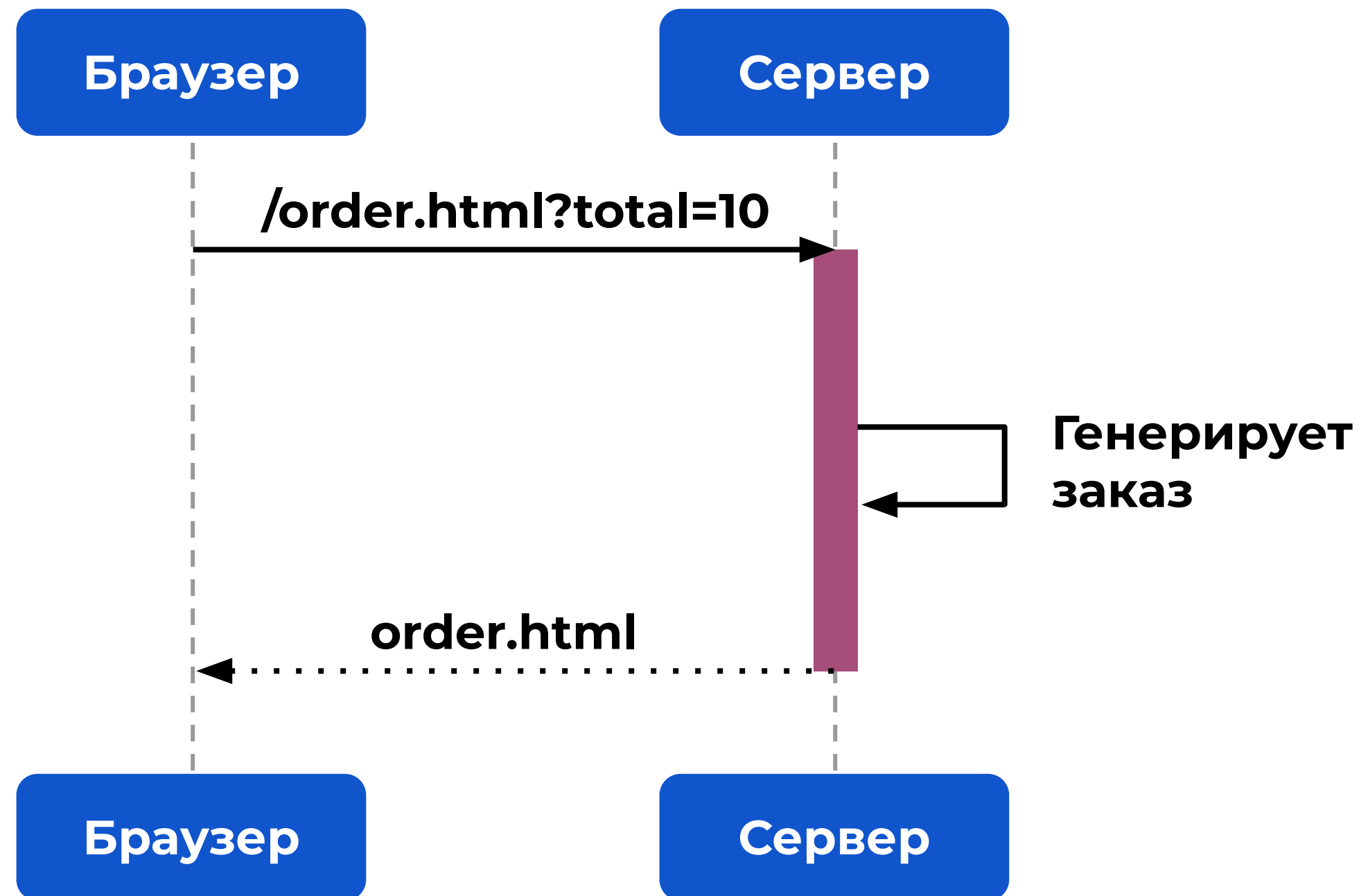


AJAX

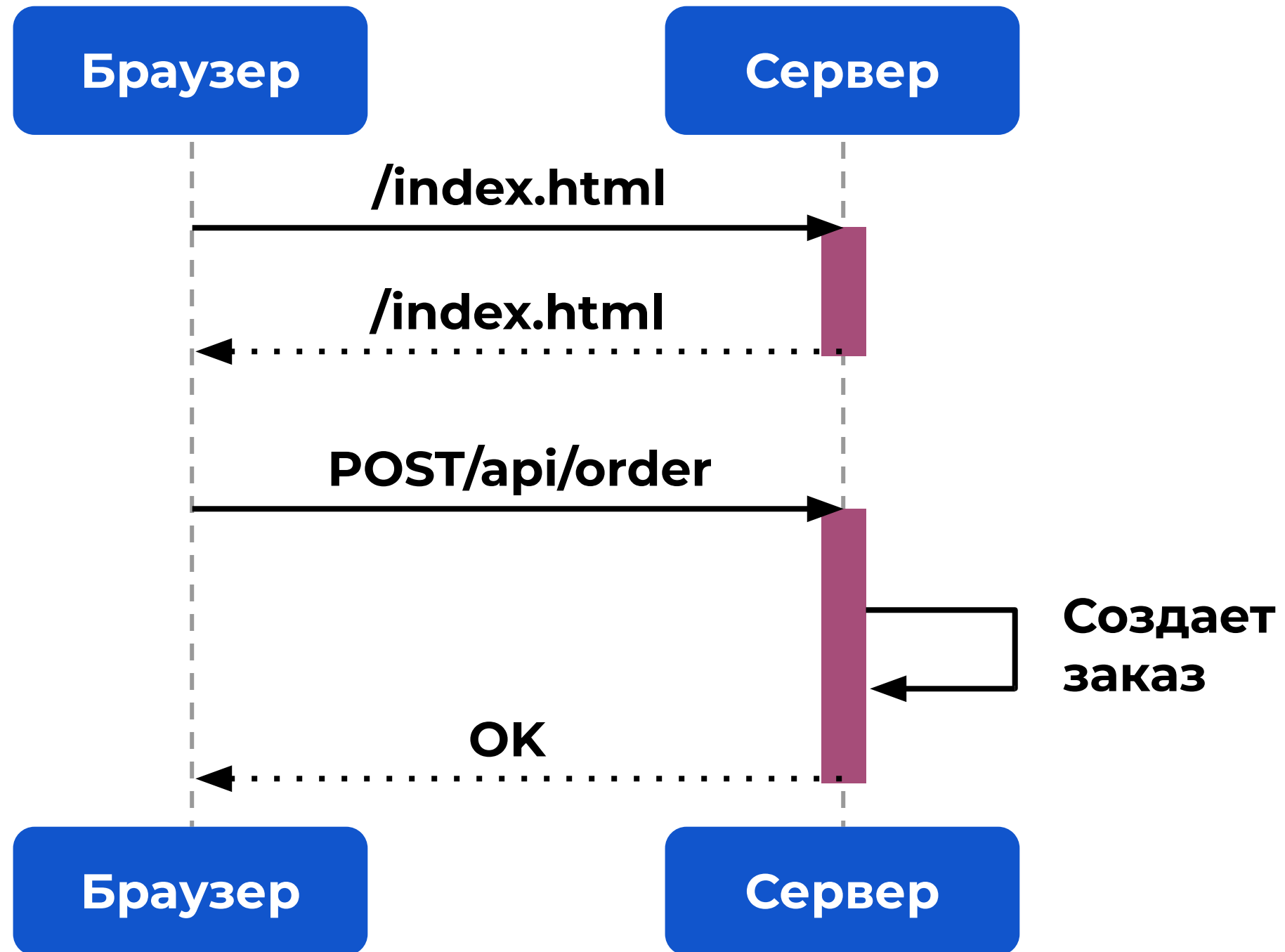
=

**Asynchronous Javascript
and XML**

Так было раньше:



А теперь так:



Перейдем к коду

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "/bar/foo.txt", true);
xhr.onload = function (e) {
    if (xhr.readyState === 4) {
        if (xhr.status === 200) {
            console.log(xhr.responseText);
        } else {
            console.error(xhr.statusText);
        }
    }
};
xhr.onerror = function (e) {
    console.error(xhr.statusText);
};
xhr.send(null);
```



Перейдем к коду

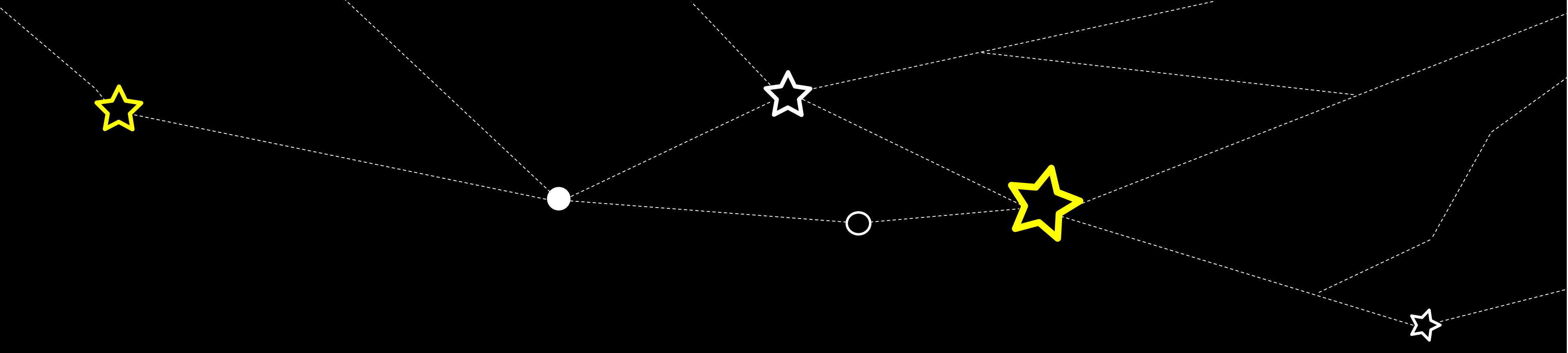
```
fetch(url, {  
  method: "POST",  
  mode: "cors",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: `{  
    "keyA": 1,  
    "keyB": 2  
  }`  
})  
.then((response) => response.json())  
.then((body) => console.log(body))  
.catch((e) => console.error(e));
```



Перейдем к коду

```
const controller = new AbortController();  
const signal = controller.signal;  
  
fetch(url, { signal })  
  
controller.abort();
```





СПАСИБО ЗА ВНИМАНИЕ

Игорь Кривонос



[ihar-kryvanos](https://www.linkedin.com/in/ihar-kryvanos)

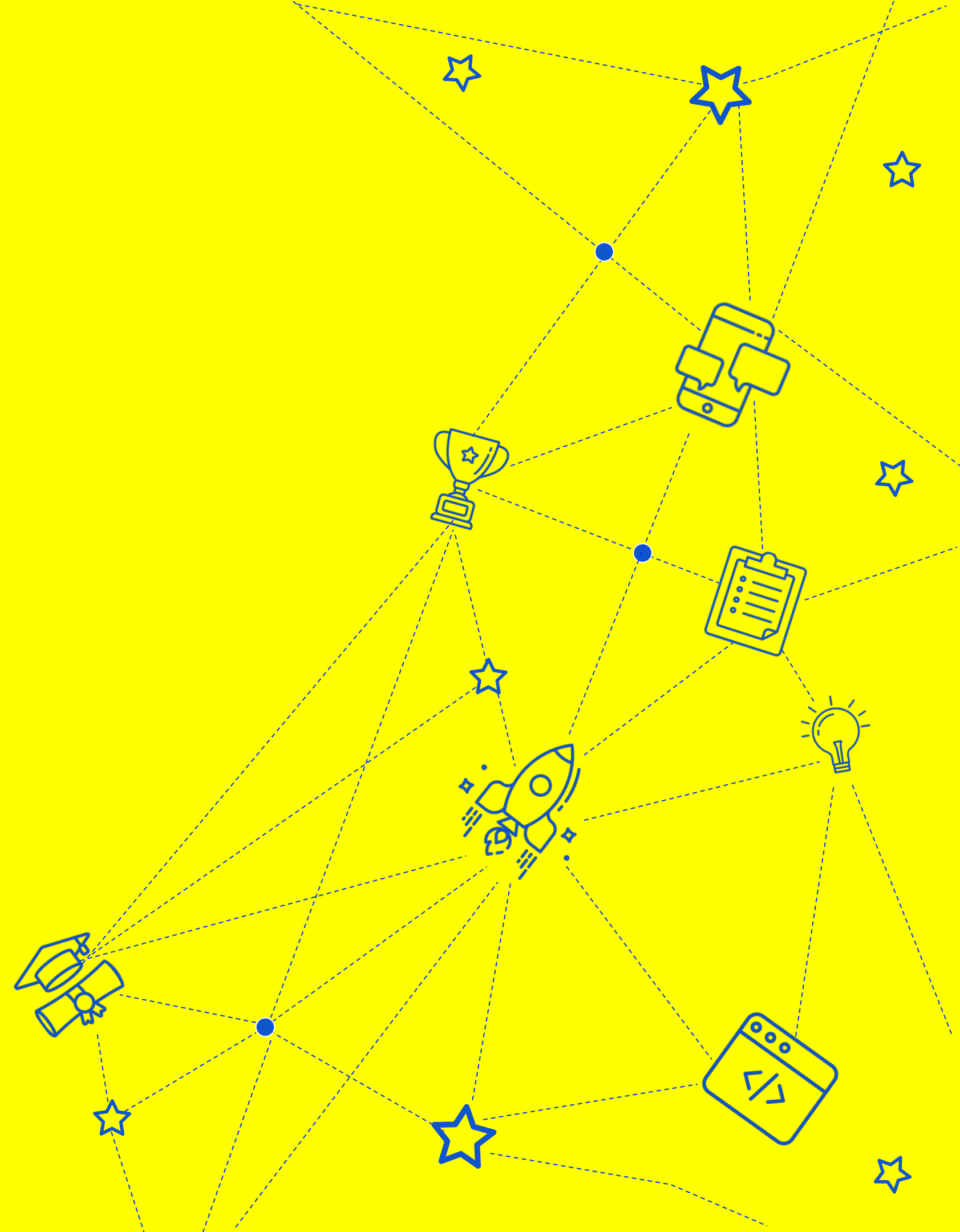


[@ikryvanos](https://www.telegram.me/ikryvanos)



[ikryvanos](https://github.com/ikryvanos)

Воркшоп



Содержание воркшопа:

- ★ Как отправить AJAX запрос
- ★ Как отправить XML/JSON на backend
- ★ Как получить ответ от сервера

